

Event Loop & Task Scheduling

JavaScript Advanced — Senior Developer Level

1. Event Loop আসলে কেন দরকার?

JavaScript execution model-এর মূল বৈশিষ্ট্য:

একটি **JavaScript execution agent** এক সময়ে একটি **JavaScript statement execute** করে।

অর্থাৎ JavaScript সাধারণভাবে একই execution thread-এ একসাথে দুইটা JavaScript function চালায় না।

তাহলে প্রশ্ন:

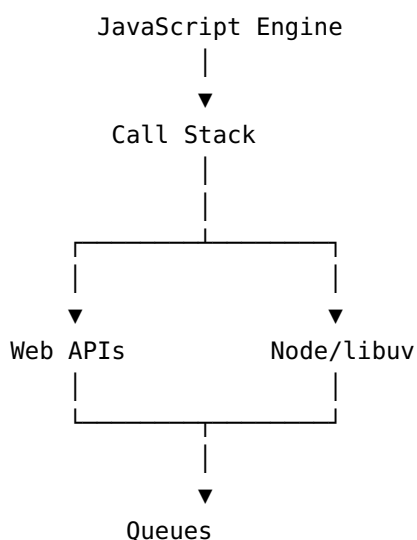
```
fetch("/users");  
  
console.log("Hello");
```

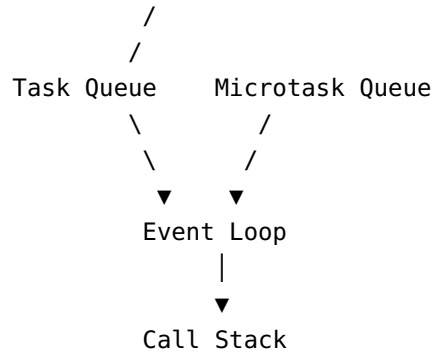
`fetch()` যদি network-এর জন্য 2 seconds নেয়, তাহলে কি পুরো JavaScript 2 seconds আটকে থাকবে?

না।

কারণ JavaScript engine এবং host environment—যেমন Browser বা Node.js—মিলে asynchronous কাজগুলো schedule করে।

একটি simplified model:





Event loop-এর মূল কাজ হলো:

কখন কোন **pending JavaScript callback execute** করা যাবে সেটা **coordinate** করা।

2. Call Stack

Call Stack হলো JavaScript-এর synchronous execution-এর জায়গা।

উদাহরণ:

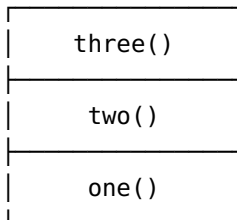
```
function one() {  
  two();  
}  
  
function two() {  
  three();  
}  
  
function three() {  
  console.log("Hello");  
}  
  
one();
```

Execution:

```
one()  
↓  
two()  
↓  
three()
```

↓
`console.log()`

Call Stack:



`three()` শেষ হলে:

```
three()
  ↓
pop

two()
  ↓
pop

one()
  ↓
pop
```

শেষে:

Call Stack = empty

3. Run-to-completion

JavaScript-এর অত্যন্ত গুরুত্বপূর্ণ property:

একটি running task/function মাঝখানে অন্য JavaScript callback এসে interrupt করতে পারে না।

উদাহরণ:

```

console.log("A");

for (let i = 0; i < 10000000000; i++) {
  // expensive work
}

console.log("B");

```

এখানে loop চলার সময় browser সাধারণ JavaScript callback execute করতে পারবে না।

অর্থাৎ:

```

Start Task
  ↓
A
  ↓
Huge loop
  ↓
B
  ↓
Task complete
  ↓
Other work gets a chance

```

এই কারণেই **CPU-heavy JavaScript main thread block** করতে পারে।

MDN-এর execution model-এ এটাকে run-to-completion model হিসেবে বর্ণনা করা হয়েছে।

4. Web APIs কী?

Browser JavaScript engine একা সব কাজ করে না।

Browser host environment অনেক API দেয়:

```

Browser
├── JavaScript Engine
├── DOM
├── Fetch
├── Timers
├── WebSocket

```

- └ Event system
- └ Rendering
- └ Storage
- └ Other Web APIs

উদাহরণ:

```
setTimeout(() => {  
  console.log("Timer");  
}, 1000);
```

JavaScript engine শুধু বসে 1 second অপেক্ষা করে না।

Conceptually:

```
JavaScript  
  |  
  | setTimeout(...)  
  ▼  
Browser Timer System  
  |  
  | wait  
  ▼  
Callback becomes runnable  
  |  
  ▼  
Task Queue  
  |  
  ▼  
Event Loop  
  |  
  ▼  
Call Stack
```

একইভাবে:

```
fetch("/users")
```

network operation browser-এর networking infrastructure handle করতে পারে এবং completion-এর পর JavaScript continuation schedule হয়।

5. Task Queue / Callback Queue

Browser environment-এ asynchronous callbacks বিভিন্ন task হিসেবে schedule হতে পারে।

উদাহরণ:

```
setTimeout(() => {  
  console.log("Timer");  
}, 0);
```

এখানে 0 মানে:

"এখনই execute করো"

না।

বরং:

"কমপক্ষে এই delay-এর আগে execute করার চেষ্টা করো না।"

তারপর callback runnable হলে event loop-এর scheduling rules অনুযায়ী execute হবে।

MDN task-কে event dispatch, timer firing ইত্যাদির মতো scheduled work হিসেবে ব্যাখ্যা করে।

6. Microtask Queue

Microtask হলো আরও high-priority ধরনের deferred JavaScript work।

প্রধান source:

```
Promise.then()  
Promise.catch()  
Promise.finally()  
queueMicrotask()  
MutationObserver
```

উদাহরণ:

```
console.log("A");
```

```
Promise.resolve().then(() => {  
  console.log("B");  
});  
  
console.log("C");
```

Output:

```
A  
C  
B
```

কারণ Promise callback synchronous execution-এর মধ্যে সরাসরি চলে না।

এটি microtask queue-তে যায়।

7. Task বনাম Microtask

একটি simplified browser model:

```
Execute Task  
  |  
  ▼  
Call Stack empty?  
  |  
  ▼  
Drain Microtask Queue  
  |  
  ▼  
Rendering opportunity  
  |  
  ▼  
Next Task
```

সবচেয়ে গুরুত্বপূর্ণ ব্যাপার:

Microtask queue empty না হওয়া পর্যন্ত event loop পরবর্তী task-এ যেতে পারে না।

উদাহরণ:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

Promise.resolve().then(() => {
  console.log("C");
});

console.log("D");
```

Output:

```
A
D
C
B
```

কারণ:

```
A
↓
schedule task B
↓
schedule microtask C
↓
D
↓
current task শেষ
↓
microtask C
↓
next task B
```

MDN-এর runtime model অনুযায়ী task-এর পর pending microtasks drain করা হয়; নতুন microtask তৈরি হলেও সেটিও current microtask checkpoint-এর মধ্যেই execute হতে পারে।

8. Microtask Queue কেন Dangerous হতে পারে?

ধরো:


```
function loop() {  
  queueMicrotask(loop);  
}  
  
loop();
```

এখানে:

```
microtask  
↓  
microtask  
↓  
microtask  
↓  
microtask  
↓  
...
```

Microtask queue কখনো empty হচ্ছে না।

ফলে browser next task-এ যেতে পারছে না।

```
Task  
↓  
Microtask  
↓  
Microtask  
↓  
Microtask  
↓  
Microtask  
↓  
∞
```

ফল:

- user input delay
- rendering delay
- timer delay
- page freeze

MDN-ও recursive microtask scheduling-এর ক্ষেত্রে event loop starvation-এর ঝুঁকি উল্লেখ করে।

9. Microtask বনাম Macrotask

"Macrotask" শব্দটি developer discussions-এ খুব common হলেও browser specification-এর formal terminology হিসেবে সাধারণত **task** বলা বেশি precise।

তাই:

```
"macrotask"
```

≈

```
"task"
```

Common task examples:

```
setTimeout(...)  
setInterval(...)  
DOM events
```

Microtasks:

```
Promise.then(...)  
queueMicrotask(...)  
MutationObserver
```

Conceptually:

Task	Microtask
তুলনামূলকভাবে lower priority	higher priority
এক task শেষ হওয়ার পর microtasks drain হতে পারে	microtask queue empty না হওয়া পর্যন্ত চলতে পারে
<code>setTimeout</code>	Promise callback
DOM events	<code>queueMicrotask()</code>
Rendering-এর আগে/পরে scheduling model-এর অংশ	সাধারণত rendering-এর আগে checkpoint

10. Important Output Question

এই code-এর output কী?

```

console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

Promise.resolve().then(() => {
  console.log("3");
});

queueMicrotask(() => {
  console.log("4");
});

console.log("5");

```

Answer:

```

1
5
3
4
2

```

কারণ:

```

Current Task
|
├─ 1
├─ schedule task 2
├─ schedule microtask 3
├─ schedule microtask 4
└─ 5

Microtask Queue
|
├─ 3
└─ 4

Next Task
└─ 2

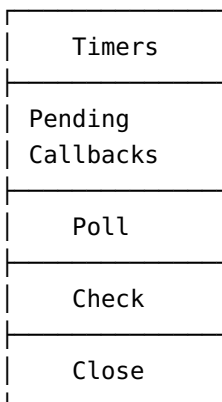
```

11. Node.js Event Loop

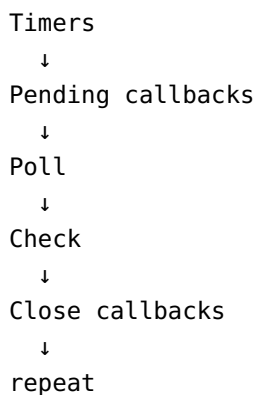
Browser event loop এবং Node.js event loop এক জিনিস না।

Node.js V8 JavaScript engine ব্যবহার করে, কিন্তু asynchronous I/O orchestration-এর জন্য libuv-এর event-driven architecture ব্যবহার করে। Node.js Event Loop এবং Worker Pool-এর মাধ্যমে কাজ ভাগ করে।

Node.js event loop-এর প্রধান phases:



একটি simplified flow:



12. Timers Phase

এখানে সাধারণত timer callbacks handle হয়:

```
setTimeout(...)  
setInterval(...)
```

উদাহরণ:

```
setTimeout(() => {  
  console.log("Timer");  
}, 100);
```

একটি common ভুল ধারণা:

```
setTimeout(fn, 100)
```

 মানে exactly 100ms পরে fn চলবে।

আসলে না।

এর অর্থ roughly:

100ms delay-এর আগে callback execute করার জন্য eligible হবে না।

তারপর event loop-এর অন্যান্য কাজের কারণে আরও delay হতে পারে।

Node.js documentation-ও timer callback-এর exact execution time guarantee করে না।

13. Pending Callbacks Phase

কিছু deferred I/O-related callback এই phase-এ process হতে পারে।

Senior-level mental model:

```
OS / libuv  
  ↓  
I/O completion  
  ↓  
callback becomes pending  
  ↓  
Node event loop processes it
```

এখানে implementation details platform এবং Node version অনুযায়ী পরিবর্তিত হতে পারে।

তাই phase names মুখস্থ করার চেয়ে কোন ধরনের কাজ কোথায় **schedule** হচ্ছে এবং কেন—এটা বোঝা বেশি গুরুত্বপূর্ণ।

14. Poll Phase

Poll phase Node.js event loop-এর খুব গুরুত্বপূর্ণ অংশ।

এখানে Node:

- নতুন I/O events দেখে
- তাদের callbacks execute করে
- প্রয়োজনে নতুন I/O-এর জন্য অপেক্ষা করে

Conceptually:

```
Poll
|
├─ socket ready?
├─ file-related completion?
├─ network event?
└─ other I/O?
```

Node.js-এর event loop OS facilities যেমন Linux-এ `epoll`, macOS-এ `kqueue`, Windows-এ IOCP ইত্যাদির সঙ্গে কাজ করে।

15. Check Phase

Check phase-এর সবচেয়ে পরিচিত API:

```
setImmediate()
```

উদাহরণ:

```
setImmediate(() => {
  console.log("Immediate");
});
```

`setImmediate()` I/O callbacks-এর পর execution schedule করার জন্য বিশেষভাবে useful।

একটি common mental model:

```
Poll
↓
```

```
Check
↓
setImmediate()
```

16. Close Callbacks

কিছু resource close-related callbacks এখানে process হতে পারে।

উদাহরণ conceptually:

```
socket.on("close", () => {
  console.log("Socket closed");
});
```

এটি resource lifecycle-এর শেষ অংশের সঙ্গে সম্পর্কিত।

17. Browser Event Loop বনাম Node.js Event Loop

এগুলোকে এক মনে করা ভুল।

Browser

```
Task
↓
Microtasks
↓
Rendering opportunity
↓
Next task
```

Node.js

```
Timers
↓
Pending callbacks
↓
Poll
↓
Check
```

```
↓  
Close  
↓  
Next iteration
```

Node.js-এর event loop phases এবং browser-এর task/rendering model আলাদা abstraction । তাই browser-এর "macrotask" diagram সরাসরি Node.js-এর phases-এর উপর বসানো উচিত নয় ।

18. process.nextTick()

Node.js-এর একটি বিশেষ scheduling mechanism:

```
process.nextTick(() => {  
  console.log("nextTick");  
});
```

এটি সাধারণ Promise microtask queue-এর সঙ্গে পুরোপুরি identical নয় ।

Node.js একটি আলাদা **next tick queue** maintain করে ।

Node documentation অনুযায়ী `process.nextTick()` current JavaScript stack শেষ হওয়ার পর এবং event loop continue করার আগে drain হয় ।

19. process.nextTick বনাম Promise

CommonJS Node.js example:

```
console.log("A");  
  
process.nextTick(() => {  
  console.log("nextTick");  
});  
  
Promise.resolve().then(() => {  
  console.log("Promise");  
});  
  
console.log("B");
```

Output:


```
A
B
nextTick
Promise
```

Conceptually:

```
Current JS
  ↓
nextTick Queue
  ↓
Promise Microtask Queue
  ↓
Event Loop
```

Node.js documentation অনুযায়ী CommonJS execution-এ `process.nextTick()` callbacks Promise/
`queueMicrotask()` callbacks-এর আগে process হয়।

20. কিন্তু ESM-এ বিষয়টা subtle

Node.js ESM module নিজেই microtask processing-এর context-এ শুরু হতে পারে।

তাই:

```
import ...
```

environment-এ একই code-এর observable ordering CommonJS-এর মতো নাও হতে পারে।

Node-এর documentation এই distinction explicitly দেখায়।

Senior developer হিসেবে rule:

```
process.nextTick()
```

 ordering নিয়ে CommonJS বনাম ESM context আলাদা হতে পারে।

তাই শুধু মুখস্থ:

```
nextTick > Promise
```

বললে পুরো picture পাওয়া যায় না।

21. process.nextTick() দিয়ে starvation

এই code:

```
function recursive() {  
  process.nextTick(recursive);  
}  
  
recursive();
```

এটি event loop-কে effectively starve করতে পারে।

কারণ:

```
nextTick  
↓  
nextTick  
↓  
nextTick  
↓  
nextTick  
↓  
...
```

Event loop অন্য phase-এ যেতে পারছে না।

Node documentation-ও recursive `process.nextTick()` ব্যবহার থেকে infinite loop-এর ঝুঁকি উল্লেখ করে।

22. queueMicrotask() কী?

Modern JavaScript-এর standard API:

```
queueMicrotask(() => {  
  console.log("Microtask");  
});
```

এটি callback-কে microtask queue-তে রাখে।

Example:

```
console.log("A");

queueMicrotask(() => {
  console.log("B");
});

console.log("C");
```

Output:

```
A
C
B
```

`queueMicrotask()` browser এবং Web Worker environments-এ widely available এবং Node.js-এও supported ।

23. Promise.then() দিয়ে Microtask বানানো বনাম queueMicrotask()

আগে অনেকে লিখত:

```
Promise.resolve().then(() => {
  console.log("microtask");
});
```

এটি কাজ করে।

কিন্তু যদি তোমার উদ্দেশ্য সরাসরি microtask schedule করা হয়:

```
queueMicrotask(() => {
  console.log("microtask");
});
```

এটি বেশি expressive ।

কারণ code দেখলেই বোঝা যায়:

এখানে Promise তৈরি করার দরকার নেই; শুধু microtask schedule করতে চাই।

MDN অনুযায়ী `queueMicrotask()` Promise-based workaround-এর কিছু overhead এবং error-handling differences এড়াতে সাহায্য করে।

24. `queueMicrotask()` কখন ব্যবহার করবে?

Useful scenario:

```
class Store {
  constructor() {
    this.listeners = [];
  }

  subscribe(listener) {
    this.listeners.push(listener);
  }

  notify() {
    queueMicrotask(() => {
      for (const listener of this.listeners) {
        listener();
      }
    });
  }
}
```

এখানে notification synchronous না রেখে current execution শেষ হওয়ার পর কিছু next task-এর আগেই করা হচ্ছে।

আরেকটি use case:

```
function emitLoaded() {
  queueMicrotask(() => {
    console.log("loaded");
  });
}
```

Library/framework implementation-এ এই ধরনের scheduling useful হতে পারে।

25. `scheduler.yield()`

Modern browser scheduling API:

```
await scheduler.yield();
```

এর উদ্দেশ্য:

দীর্ঘ-running task-এর মাঝখানে browser-কে control ফিরিয়ে দেওয়া।

উদাহরণ:

```
async function processLargeData(data) {  
  for (const item of data) {  
    process(item);  
  
    await scheduler.yield();  
  }  
}
```

Conceptually:

```
Process chunk  
  ↓  
yield  
  ↓  
Browser gets opportunity  
  ↓  
User input / rendering / other work  
  ↓  
Continue  
  ↓  
Process next chunk
```

MDN অনুযায়ী `scheduler.yield()` একটি Promise return করে এবং continuation-কে পরে prioritized task হিসেবে schedule করে।

26. scheduler.yield() বনাম queueMicrotask()

এটা খুব গুরুত্বপূর্ণ।

queueMicrotask()

```
queueMicrotask(() => {  
  heavyWork();  
});
```

এতে তুমি microtask queue-তে ঢুকছো।

Microtasks drain না হওয়া পর্যন্ত next task-এর সুযোগ নাও আসতে পারে।

scheduler.yield()

```
await scheduler.yield();  
  
heavyWork();
```

এতে browser-কে task boundary-এর সুযোগ দেওয়া হচ্ছে।

তাই:

```
queueMicrotask()  
  ↓  
"আমাকে current task-এর শেষের দিকে খুব তাড়াতাড়ি চালাও"  
  
scheduler.yield()  
  ↓  
"আমাকে পরে চালাও; browser-কে আগে কিছু work করার সুযোগ দাও"
```

এগুলো opposite scheduling goals-এর মতো ভাবতে পারো।

27. scheduler.yield() দিয়ে Long Task ভাঙা

ধরো:

```
function hugeCalculation() {  
  for (let i = 0; i < 10000000000; i++) {  
    calculate(i);  
  }  
}
```

এটি main thread block করতে পারে।

Better conceptual approach:

```
async function hugeCalculation(items) {
  for (const item of items) {
    calculate(item);

    if (shouldYield()) {
      await scheduler.yield();
    }
  }
}
```

অথবা chunks:

```
async function process(items) {
  const CHUNK_SIZE = 1000;

  for (let i = 0; i < items.length; i += CHUNK_SIZE) {
    const chunk = items.slice(i, i + CHUNK_SIZE);

    processChunk(chunk);

    await scheduler.yield();
  }
}
```

এটি **cooperative scheduling**।

28. scheduler.yield() সব browser-এ একইভাবে available নয়

এই API modern এবং এখনও browser support-এর ক্ষেত্রে universal নয়।

তাই production code-এ feature detection করা যায়:

```
if (globalThis.scheduler?.yield) {
  await scheduler.yield();
} else {
```

```
    await new Promise(resolve => setTimeout(resolve, 0));
  }
```

Current MDN documentation এটিকে non-Baseline feature হিসেবে চিহ্নিত করছে।

29. requestAnimationFrame()

Browser rendering-এর সঙ্গে কাজ করার জন্য:

```
requestAnimationFrame(() => {
  // animation/update
});
```

এটি সাধারণত browser-এর পরবর্তী rendering opportunity-এর সঙ্গে animation work synchronize করার জন্য ব্যবহার করা হয়।

Example:

```
function animate() {
  box.style.transform = `translateX(${x}px)`;

  x += 1;

  requestAnimationFrame(animate);
}

requestAnimationFrame(animate);
```

Conceptually:

```
JavaScript
  ↓
requestAnimationFrame()
  ↓
Browser prepares next frame
  ↓
callback
  ↓
render
```

Animation-এর জন্য সাধারণত:


```
setTimeout(...)
```

এর চেয়ে `requestAnimationFrame()` বেশি appropriate।

30. কেন `requestAnimationFrame()`?

ধরো display প্রায়:

60 FPS

তাহলে ideal frame budget roughly:

$$1000 / 60 \approx 16.67\text{ms}$$

অর্থাৎ প্রতিটি frame-এর জন্য browser-এর হাতে খুব সীমিত সময় থাকে।

যদি JavaScript অনেক সময় নেয়:

Frame 1



↑
JS too slow

তাহলে:

- animation jank
- dropped frames
- scrolling lag
- input delay

হতে পারে।

31. `requestIdleCallback()`

আরেকটি browser API:

```
requestIdleCallback(() => {  
  doBackgroundWork();  
});
```

এর উদ্দেশ্য:

Browser যখন relatively idle থাকবে তখন low-priority work চালানো।

Useful কাজ:

```
analytics preparation  
prefetch preparation  
non-critical calculations  
background bookkeeping
```

উদাহরণ:

```
requestIdleCallback(() => {  
  prepareAnalyticsPayload();  
});
```

MDN অনুযায়ী এটি browser-এর idle periods-এ low-priority কাজ schedule করে।

32. requestIdleCallback()-এর timeout

একটি সমস্যা:

```
requestIdleCallback(() => {  
  doWork();  
});
```

যদি browser কখনো idle না হয়?

তাহলে callback অনেক delay করতে পারে।

তাই required background work হলে:

```
requestIdleCallback(  
  () => {
```

```

    doWork();
  },
  {
    timeout: 2000
  }
);

```

এতে browser-কে বলা হচ্ছে:

idle opportunity না পেলেও deadline-এর কাছাকাছি এসে কাজটা করতে হবে।

MDN timeout option ব্যবহার করার পরামর্শ দেয় কারণ timeout ছাড়া callback দীর্ঘ সময় অপেক্ষা করতে পারে।

33. requestAnimationFrame বনাম requestIdleCallback

API	উদ্দেশ্য
<code>requestAnimationFrame()</code>	visual update / animation
<code>requestIdleCallback()</code>	low-priority background work
<code>setTimeout()</code>	general delayed task
<code>queueMicrotask()</code>	microtask scheduling
<code>scheduler.yield()</code>	current task ভেঙে browser-কে সুযোগ দেওয়া

Mental model:

```

Need animation?
  ↓
requestAnimationFrame()

Need low-priority background work?
  ↓
requestIdleCallback()

Need microtask?
  ↓
queueMicrotask()

Need to yield long-running work?

```

```
↓  
scheduler.yield()
```

34. Long Tasks কী?

Browser main thread-এ যদি একটি task দীর্ঘ সময় uninterrupted চলে, তাকে long task বলা হয়।

Web Performance APIs-এ সাধারণত:

50ms+

এর মতো uninterrupted main-thread work long task হিসেবে পর্যবেক্ষণ করা হয়।

Long task-এর সমস্যা:

```
User clicks  
↓  
Main thread busy  
↓  
No immediate response  
↓  
Input delay
```

এছাড়া:

- animation jank
- scrolling lag
- delayed rendering
- poor responsiveness

হতে পারে।

35. Main Thread Blocking

এই code dangerous:

```
button.addEventListener("click", () => {  
  const start = performance.now();
```

```

while (performance.now() - start < 5000) {
  // block main thread
}
});

```

এই 5 seconds:

JavaScript



Input	×
Rendering	×
Animation	×
Other JS	×

Browser-এর main thread basically occupied ।

36. Node.js-এ Long Task-এর সমস্যা

Node.js server-এ:

```

app.get("/data", (req, res) => {
  expensiveCalculation();

  res.json(result);
});

```

যদি:

expensiveCalculation()

অনেক CPU time নেয়, তাহলে Event Loop অন্য request-এর callback execute করতে দেরি করবে ।

Conceptually:

Request A

↓

Huge CPU work

↓



↓
Request B waits
Request C waits
Request D waits

Node.js-এর official guidance অনুযায়ী Event Loop callback ছোট রাখা গুরুত্বপূর্ণ; CPU-heavy কাজ প্রয়োজনে partition বা offload করা উচিত।

37. Partitioning

একটি বড় কাজ:

```
for (let i = 0; i < 1000000000; i++) {  
  work(i);  
}
```

এর পরিবর্তে ছোট chunks:

```
Chunk 1  
↓  
yield  
↓  
Chunk 2  
↓  
yield  
↓  
Chunk 3  
↓  
yield
```

Node.js-এর official guidance-এও expensive calculation partition করার ধারণা দেওয়া হয়েছে।

Browser-এ:

```
await scheduler.yield();
```

Node.js-এ use case অনুযায়ী:

```
setImmediate(...)
```

বা Worker Threads ব্যবহার করা যেতে পারে।

38. Partitioning বনাম Offloading

এ দুটো আলাদা।

Partitioning

একই thread-এ কাজকে ছোট অংশে ভাগ করা।

```
Main Thread
|
├─ chunk
├─ yield
├─ chunk
├─ yield
└─ chunk
```

Offloading

কাজ অন্য thread-এ পাঠানো।

```
Main Event Loop
|
▼
Worker Thread
|
▼
CPU-heavy calculation
```

যদি কাজ সত্যিই CPU-heavy হয়:

শুধু `yield()` করলেই CPU cost disappear করবে না।

Worker Thread বেশি appropriate হতে পারে।

39. Event Loop Lag

Event loop lag মানে:

Event loop যে সময়ে কাজ execute করার কথা ছিল তার তুলনায় বাস্তবে কত দেরি হচ্ছে।

ধরো:

```
setTimeout(() => {  
  console.log("done");  
}, 100);
```

Expected:

~100ms+

কিন্তু main thread যদি busy থাকে:

```
100ms  
  ↓  
200ms  
  ↓  
500ms
```

তাহলে বুঝতে পারো event loop busy ছিল।

40. Simple Event Loop Lag Measurement

Node.js-এ একটি basic approach:

```
const interval = 100;  
  
setInterval(() => {  
  const expected = Date.now() + interval;  
  
  setTimeout(() => {  
    const lag = Date.now() - expected;  
  
    console.log("Lag:", lag, "ms");  
  }, interval);  
, interval);
```

তবে এটি production-grade monitoring solution নয়।

কারণ scheduling itself-এর অনেক nuance আছে।

41. Better Node.js Monitoring

Node.js ecosystem-এ event loop utilization/lag monitor করার জন্য runtime performance APIs ব্যবহার করা যায়।

Conceptually:

```
Application
  ↓
Event Loop
  ↓
Measure responsiveness
  ↓
Metrics
  ↓
Monitoring system
```

Important metrics:

```
Event loop delay
Event loop utilization
CPU usage
GC pauses
Request latency
p95 / p99 latency
```

Senior backend developer হিসেবে শুধু average latency দেখলে হবে না।

বিশেষ করে:

```
p95
p99
```

দেখা গুরুত্বপূর্ণ।

42. Browser-এ Long Task Monitoring

Browser PerformanceObserver দিয়ে long task observe করা যায়।

Example:

```
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    console.log("Long task:", entry.duration);
  }
});

observer.observe({
  type: "longtask",
  buffered: true
});
```

Long Tasks API main UI thread-এর 50ms বা তার বেশি uninterrupted work detect করতে পারে, যদিও browser support সব জায়গায় universal নয়।

43. Microtask Starvation বনাম Long Task

দুটো দেখতে একই ধরনের problem তৈরি করতে পারে।

Long synchronous task

```
while (true) {
  // heavy synchronous work
}
```

Microtask starvation

```
function loop() {
  queueMicrotask(loop);
}

loop();
```

দুটোর ক্ষেত্রেই:

Other work delayed
↓
UI responsiveness ↓

কিন্তু root cause আলাদা।

44. একটি Complete Browser Example

```
console.log("1");

setTimeout(() => {
  console.log("2");
}, 0);

queueMicrotask(() => {
  console.log("3");

  queueMicrotask(() => {
    console.log("4");
  });
});

Promise.resolve().then(() => {
  console.log("5");
});

requestAnimationFrame(() => {
  console.log("6");
});

console.log("7");
```

Basic expected ordering:

```
1
7
3
5
4
...
2 / 6
```

কিন্তু এখানে একটি গুরুত্বপূর্ণ senior-level warning:

`requestAnimationFrame()` এবং task execution-এর exact relative ordering-কে
`setTimeout()`-এর মতো simple universal rule হিসেবে ধরে নেওয়া উচিত নয়।

Browser rendering opportunity, frame timing এবং অন্যান্য pending work-এর ওপর behavior নির্ভর করতে পারে।

45. একটি Complete Node.js Example

CommonJS context:

```
console.log("A");

setTimeout(() => {
  console.log("B");
}, 0);

setImmediate(() => {
  console.log("C");
});

Promise.resolve().then(() => {
  console.log("D");
});

process.nextTick(() => {
  console.log("E");
});

console.log("F");
```

সাধারণভাবে:

```
A
F
E
D
...
```

এরপর timer/immediate-এর relative ordering context-এর ওপর নির্ভর করতে পারে।

এখানে senior lesson:

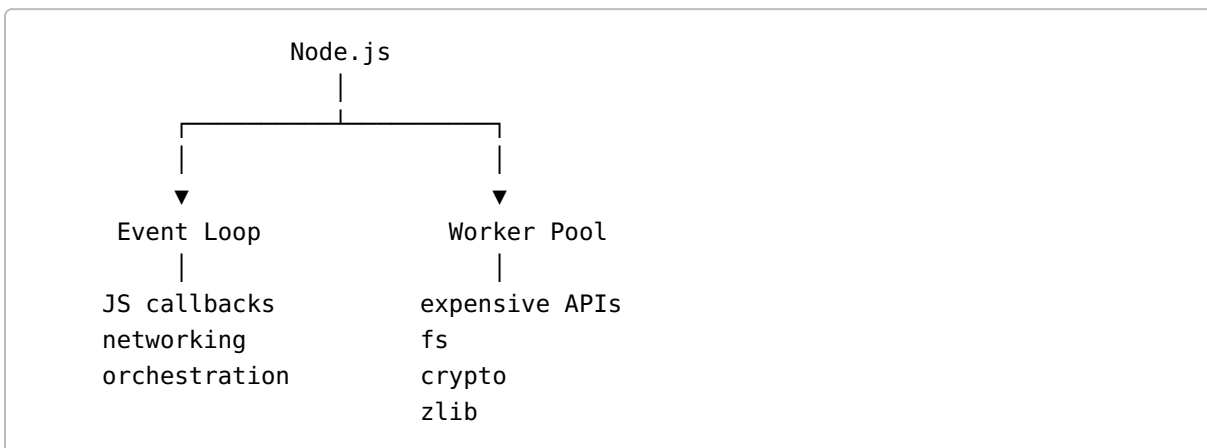
`setTimeout(..., 0)` বনাম `setImmediate()` নিয়ে সব context-এ একটি fixed universal output মুখস্থ করা ঠিক নয়।

বিশেষ করে I/O callback-এর ভিতরে ordering আলাদা হতে পারে।

46. Node.js Worker Pool ভুলে যেও না

Node.js শুধু Event Loop না।

Conceptually:



Node.js-এর libuv Worker Pool কিছু expensive operations handle করে, যেমন কিছু filesystem, DNS, crypto এবং zlib operations।

তাই:

"Node.js single-threaded"

বলে incomplete picture হয়।

Better:

JavaScript execution on the main Node.js event-loop thread is single-threaded, while Node.js itself uses other threads/resources for certain operations.

47. `await` Event Loop-এর সঙ্গে কীভাবে সম্পর্কিত?

ধরো:

```
async function test() {  
  console.log("A");  
  
  await Promise.resolve();  
  
  console.log("B");  
}
```

```
}  
  
test();  
  
console.log("C");
```

Output:

```
A  
C  
B
```

কারণ:

```
test()  
↓  
A  
↓  
await  
↓  
function suspends  
↓  
C  
↓  
microtask  
↓  
B
```

`await` asynchronous continuation তৈরি করে এবং continuation সাধারণত Promise-based microtask scheduling-এর সঙ্গে যুক্ত।

48. `await` মানে **Thread Sleep** না

খুব গুরুত্বপূর্ণ:

```
await fetch("/users");
```

মানে:

JavaScript thread sleep করছে

এমন না।

বরং:

```
async function
  ↓
await
  ↓
function continuation suspended
  ↓
event loop can run other work
  ↓
Promise settles
  ↓
continuation scheduled
```

এই কারণেই `await` ব্যবহার করেও Node.js concurrent I/O handle করতে পারে।

49. Microtask Queue-এর Senior Mental Model

এটা মনে রাখো:

```
Current synchronous execution
  ↓
Current task
  ↓
Stack becomes empty
  ↓
Microtask checkpoint
  ↓


Promise callbacks  

    queueMicrotask()  

    other microtasks


  ↓
Rendering / next  
scheduling step
```

সব microtasks drain না হওয়া পর্যন্ত এগিয়ে যাওয়া যাবে না।

50. Scheduling APIs — কখন কোনটা?

Situation	API
Immediate synchronous execution	normal function
General delayed work	<code>setTimeout()</code>
Node I/O-এর পর কাজ	<code>setImmediate()</code>
Promise-style microtask	<code>queueMicrotask()</code>
Promise continuation	<code>.then()</code> / <code>await</code>
Browser animation	<code>requestAnimationFrame()</code>
Low-priority browser work	<code>requestIdleCallback()</code>
Break up long browser task	<code>scheduler.yield()</code>
CPU-heavy Node/browser work	Worker / Worker Thread

51. Common Mistake #1

ভুল:

```
setTimeout(fn, 0);
```

মানে:

fn এখনই চলবে

সঠিক:

fn earliest eligible task হিসেবে schedule হবে

কিন্তু:

```
current task  
+  
microtasks
```


+
other scheduling

এর কারণে execution পরে হতে পারে।

52. Common Mistake #2

ভুল:

Promise asynchronous, তাই Promise callback সবসময় অন্য async কাজের পরে চলে।

না।

Promise callback microtask হওয়ায় অনেক task-এর আগে execute হতে পারে।

```
setTimeout(() => console.log("timer"), 0);

Promise.resolve().then(() => {
  console.log("promise");
});
```

সাধারণ browser model:

```
promise
timer
```

53. Common Mistake #3

ভুল:

`queueMicrotask()` performance improve করে।

সবসময় না।

বরং excessive microtasks:

```
queueMicrotask(...)
queueMicrotask(...)
```

```
queueMicrotask(...)  
...
```

browser-কে অন্য কাজের সুযোগ না দিয়ে starvation তৈরি করতে পারে।

Microtask হলো **ordering/scheduling tool**, performance magic নয়।

54. Common Mistake #4

ভুল:

async function মানেই background thread।

না।

```
async function test() {  
  heavyCPUWork();  
}
```

এখানে:

```
heavyCPUWork()
```

still main JavaScript thread-এ চলবে।

`async` keyword CPU work-কে magically অন্য thread-এ পাঠায় না।

55. Common Mistake #5

ভুল:

Node.js single-threaded তাই কোনো কাজ parallel হয় না।

Node.js architecture-এ:

```
Event Loop  
+  
Worker Pool  
+
```

OS async facilities
+
Worker Threads

সব মিলিয়ে concurrency তৈরি হতে পারে।

56. Long Task Debugging Workflow

Browser app slow হলে:

Step 1

Performance timeline দেখো।

Step 2

Long task খুঁজো।

Step 3

দেখো কোন JS function বেশি সময় নিচ্ছে।

Step 4

চেক করো:

Can it be optimized?
Can it be chunked?
Can it be yielded?
Can it be moved to Worker?

Step 5

UI update এবং computation আলাদা করো।

57. Node.js Event Loop Debugging Workflow

API slow হলে:

```
Request latency high
  ↓
CPU high?
  ↓
Event Loop blocked?
  ↓
Large JSON?
  ↓
Regex?
  ↓
Sync fs?
  ↓
Crypto?
  ↓
Large loop?
  ↓
Worker Pool saturated?
```

বিশেষ করে server-side code-এ:

```
fs.readFileSync()
JSON.parse(hugeData)
JSON.stringify(hugeObject)
crypto.*Sync()
large loops
```

এসব carefully evaluate করা উচিত।

Node.js-এর official guidance Event Loop blocking-এর জন্য synchronous expensive APIs এবং large JSON processing-এর ঝুঁকি উল্লেখ করে।

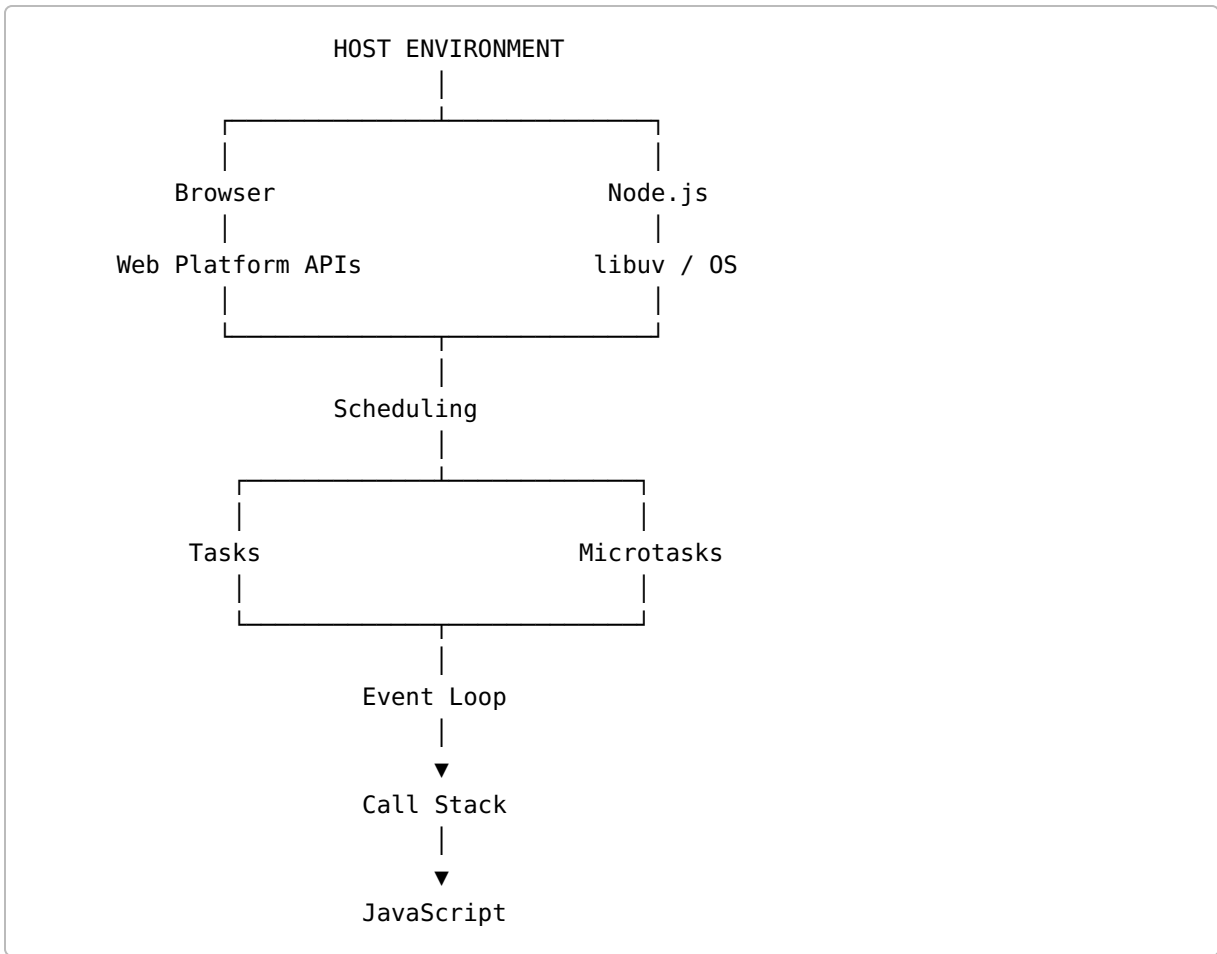
58. Senior-Level Mental Model

Event Loop-কে শুধু:

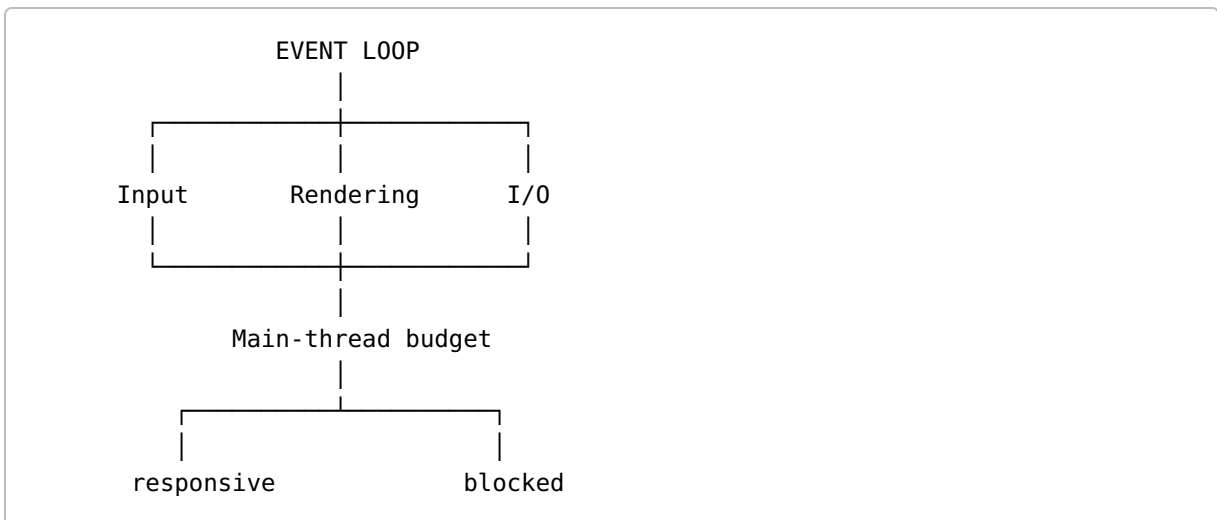
```
Call Stack → Queue → Call Stack
```

এভাবে দেখলে Advanced level-এর জন্য যথেষ্ট না।

Better model:



আর performance perspective:



59. সবচেয়ে গুরুত্বপূর্ণ 10টি Rule

Rule 1

JavaScript execution is run-to-completion.

Rule 2

Call Stack synchronous execution manage করে।

Rule 3

Browser APIs asynchronous operations facilitate করে।

Rule 4

Task এবং microtask এক জিনিস নয়।

Rule 5

Microtasks সাধারণত current task-এর পর drain হয়।

Rule 6

Microtask queue recursively grow করলে starvation হতে পারে।

Rule 7

`process.nextTick()` Node-specific scheduling mechanism।

Rule 8

`queueMicrotask()` standard microtask scheduling API।

Rule 9

`requestAnimationFrame()` visual work-এর জন্য।

Rule 10

CPU-heavy কাজ হলে শুধু `async/await` যথেষ্ট নয়—partition বা Worker দরকার হতে পারে।

60. Quick Revision Table

Topic	Core idea
Call Stack	synchronous JS execution
Web APIs	browser-provided async capabilities
Task	scheduled unit of work
Microtask	higher-priority deferred work
Promise	microtask continuation তৈরি করতে পারে
<code>queueMicrotask()</code>	directly microtask schedule করে
Node Event Loop	Node-specific phase-based scheduling
<code>process.nextTick()</code>	Node next-tick queue
<code>scheduler.yield()</code>	long task থেকে cooperative yield
<code>requestAnimationFrame()</code>	frame-synchronized visual work
<code>requestIdleCallback()</code>	low-priority idle work
Long Task	দীর্ঘ uninterrupted main-thread work
Event Loop Lag	scheduling/execution delay
Partitioning	বড় কাজকে ছোট অংশে ভাগ করা
Offloading	অন্য thread/worker-এ কাজ পাঠানো

61. Practice Problems

Problem 1

Output predict করো:

```
console.log("A");

setTimeout(() => console.log("B"), 0);

queueMicrotask(() => console.log("C"));

Promise.resolve().then(() => console.log("D"));
```

```
console.log("E");
```

Problem 2

Explain:

```
queueMicrotask(() => {  
  queueMicrotask(() => {  
    console.log("A");  
  });  
  
  console.log("B");  
});  
  
console.log("C");
```

Output:

```
C  
B  
A
```

কেন?

Problem 3

এই code-এর problem কী?

```
function forever() {  
  queueMicrotask(forever);  
}  
  
forever();
```

Problem 4

এই দুইটার পার্থক্য explain করো:


```
queueMicrotask(work);
```

vs

```
setTimeout(work, 0);
```

Problem 5

কখন:

```
scheduler.yield()
```

ব্যবহার করবে এবং কখন:

```
Worker
```

ব্যবহার করবে?

Problem 6

Node.js server-এ:

```
app.get("/users", async (req, res) => {  
  const users = hugeArray.map(expensiveCalculation);  
  
  res.json(users);  
});
```

কীভাবে Event Loop block হতে পারে explain করো।

62. Interview Questions

Junior → Mid

1. What is the event loop?
2. What is the call stack?
3. What is a task queue?

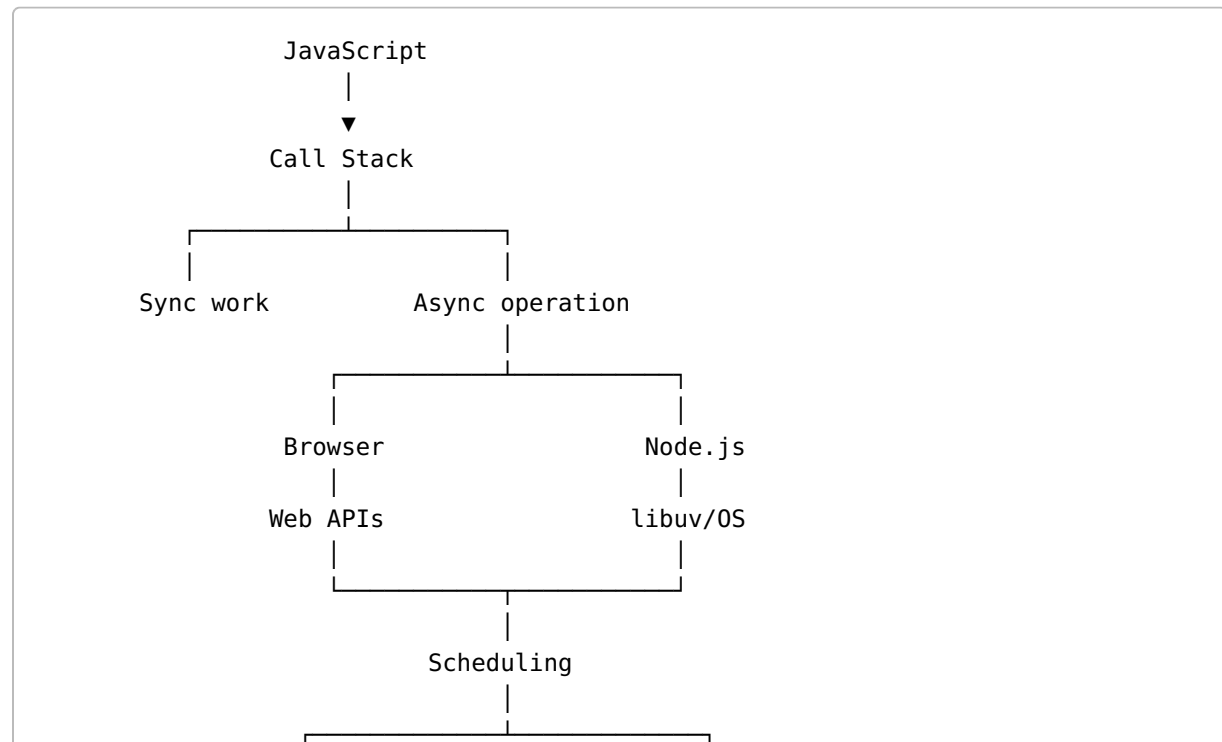
4. What is a microtask?
5. Why does Promise callback run before `setTimeout(..., 0)`?

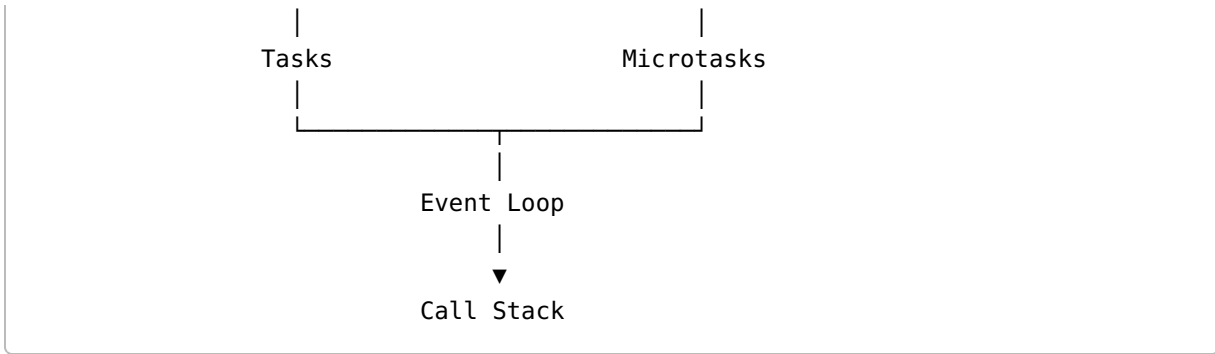
Mid → Senior

1. Explain task vs microtask.
2. Why can microtasks cause starvation?
3. Explain Node.js event loop phases.
4. `process.nextTick()` vs Promise microtask?
5. Why doesn't `async` automatically make CPU work non-blocking?
6. What is event loop lag?
7. How would you detect long tasks?
8. `queueMicrotask()` কখন use করবে?
9. `requestAnimationFrame()` কেন animation-এর জন্য appropriate?
10. `requestIdleCallback()` কখন ব্যবহার করা উচিত?
11. `scheduler.yield()` কী problem solve করে?
12. Partitioning vs offloading?
13. Why can a Node.js server become slow even if all APIs are asynchronous?
14. How can a microtask queue starve the event loop?
15. How would you debug a production event-loop bottleneck?

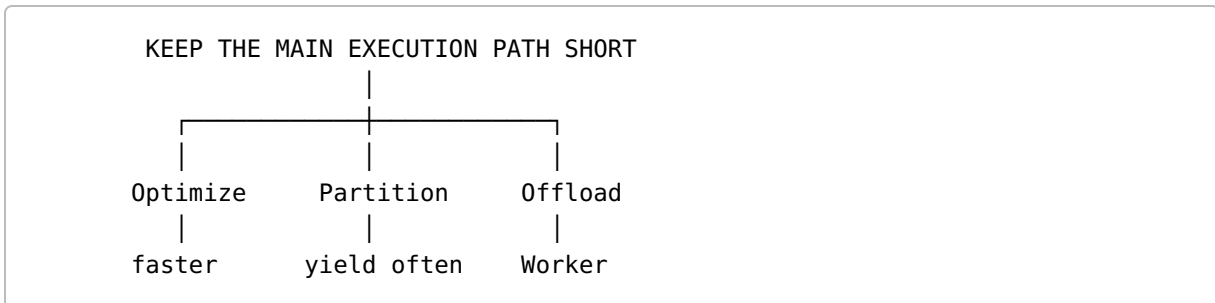
63. Final Mental Model

শেষে পুরো chapter-টা এইভাবে মনে রাখো:





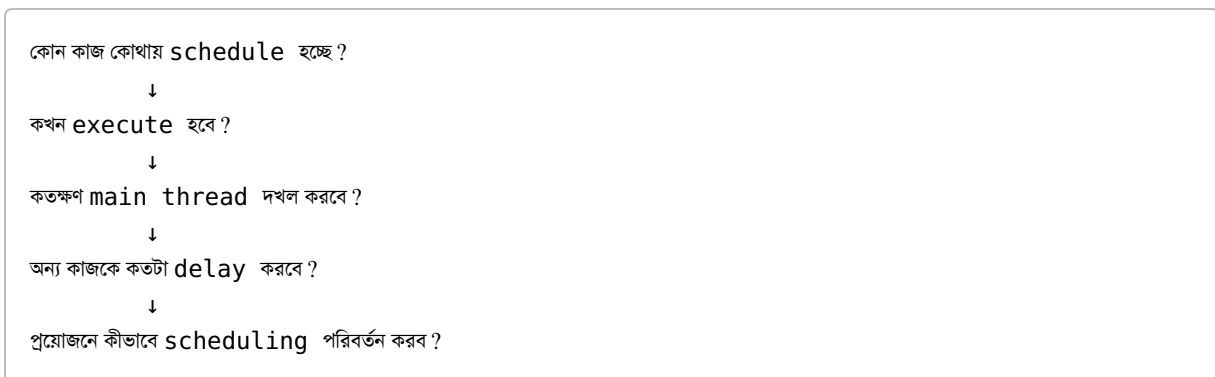
আর performance-এর দিক থেকে:



সবচেয়ে বড় lesson:

Event Loop শেখার উদ্দেশ্য শুধু **output predict** করা না।

Senior developer হিসেবে তোমার বুঝতে হবে:



এই mental model-টাই browser performance, Node.js scalability এবং production debugging—তিন জায়গাতেই সবচেয়ে বেশি কাজে লাগবে।